

EduConnect — Developer Handover Documentation

This document describes the **server** (Express + Prisma) and **web client** (React + Vite). The mobile workspace exists in `mobile/` but is intentionally out of scope here.

1. Overview

EduConnect is an education marketplace connecting international students with Australian universities, education agents, and consultants. The core feature surface is:

- **Authentication & roles** — five roles: `ADMIN`, `UNIVERSITY`, `AGENT`, `CONSULTANT`, `STUDENT`. Role discriminates the User row in a single table; each non-admin role has a 1:1 profile table.
 - **Marketplace directories** — public listings for universities, agents, consultants. Includes a "compare universities" batch lookup.
 - **Posts / community feed** — categorised posts with upvotes, bookmarks, and threaded comments. Authors are users of any role.
 - **Enquiries (Leads)** — students send an enquiry to a university / agent / consultant. The server persists a `Lead` and sends an **email** to the recipient with the student's message. Status flow: `NEW` → `CONTACTED` → `CONVERTED` / `CLOSED`.
 - **Reviews** — students post star + body reviews on a target user (uni / agent / consultant). Each review recomputes the target's `rating` and `reviewCount`.
 - **Campaigns** — universities own marketing campaigns with start/end dates and `ACTIVE/PAUSED/ENDED` status.
 - **Admin panel** — list users, approve / suspend / reactivate non-student accounts, pin / hide / remove posts, dashboard counts.
-

2. Tech stack

Server (`server/`)

- **Runtime:** Node.js 20+, ES modules, `node --watch` for dev.
- **Framework:** Express.
- **ORM / DB:** Prisma 6 + PostgreSQL.
- **Auth:** JWT access + opaque rotating refresh tokens (httpOnly signed cookies for web; `Authorization: Bearer` for mobile).
- **Password hashing:** Argon2id (OWASP-recommended params).
- **Validation:** zod schemas applied via a `validate` middleware.
- **Security:** Helmet (CSP, HSTS, frameguard, noSniff), CORS allow-list, CSRF (double-submit via `csrf-csrf`), `hpp` (HTTP parameter pollution), rate limits (`express-rate-limit`).
- **Email:** `nodemailer` — uses configured SMTP if `SMTP_*` env vars are set, otherwise falls back to a free `Ethereal` test inbox in dev.
- **Logging:** Pino + `pino-http`; pretty-print in dev, redaction of secrets.
- **Cron:** `node-cron` for background jobs.

Client (`client/`)

- **Framework:** React 19 + Vite + Tailwind CSS v4.
 - **Routing:** `react-router-dom` v6 with lazy-loaded routes.
 - **State:** Zustand (auth-only; all domain data is fetched per-page).
 - **HTTP:** custom fetch wrapper with CSRF token caching, silent refresh on 401, CSRF rotation on 403.
 - **UI bits:** `framer-motion`, `lucide-react`, `recharts`, `react-hot-toast`.
-

3. Repository layout

client/	React + Vite SPA
server/	Express API + Prisma
mobile/	Expo React Native (out of scope here)
prisma/	(lives under server/) schema + migrations + seed
docs/	handover docs
README.md	first-run setup

Server source layout

server/src/	
app.js	Express app builder (middleware order, /api mount, error handlers)
server.js	HTTP server boot, cron start, graceful shutdown
config/	
env.js	zod-validated env schema (fails boot on missing vars)
logger.js	Pino logger with redaction
db/	
prisma.js	singleton PrismaClient
jobs/	
cron.js	expired refresh-token cleanup, campaign auto-end
modules/	feature folders – one per domain
index.js	aggregates module routers under /api
auth/	auth.{controller,routes,schema,service}.js
posts/	posts.{controller,routes}.js + post.schema.js
directory/	directory.{controller,routes}.js + profile.schema.js
business/	campaigns/leads/reviews controllers + business.{routes,schema}.js
admin/	admin.{controller,routes}.js
shared/	cross-cutting concerns
middleware/	auth, csrf, cors, errorHandler, securityHeaders, rateLimits, validate
utils/	tokens, cookies, asyncHandler, password, errors
services/	email (nodemailer)
validators/	common.schema (idParam, paginationQuery, emptyToUndef helper)

Client source layout

client/src/	
App.jsx	Routes, ProtectedRoute, RequireAuth, RedirectIfAuthed wrappers
api/	
client.js	fetch wrapper (CSRF, refresh, error mapping)
endpoints.js	typed API surface – one object per server module
mappers.js	UPPER_SNAKE ↔ lower-kebab + nested→flat normalisers
store/	
useStore.js	auth-only Zustand store
hooks/	
useApiResource.js	generic data-fetching hook
components/	
layout/	DashboardLayout, Navbar, Sidebar, Footer
pages/	routed screens, grouped by role/feature
auth/	Login, Signup
admin/, university/, agent/, consultant/, student/	

marketplace/ Universities, Agents, Consultants, *Detail
feed/, dashboard/MyPosts/, static/

4. Environment configuration

Validated in `server/src/config/env.js` (zod). The server **fails to boot** if any required value is missing or malformed.

Variable	Type / format	Purpose
NODE_ENV	development test production (default development)	Toggles dev-only behaviour (pretty logs, looser cookie SameSite, error stack)
PORT	int (default 5000)	HTTP listener port; project conventionally uses 5050
CLIENT_URL	URL	Allow-listed origin for CORS
TRUST_PROXY	true / false (default false)	Set <code>app.set('trust proxy', 1)</code> so <code>req.ip</code> works behind <code>nginx</code> /ALB
DATABASE_URL	string ≥ 10 chars	Postgres connection URL
COOKIE_SECRET	string ≥ 32 chars	Signs the access/refresh cookies
JWT_ACCESS_SECRET	string ≥ 32 chars	HS256 signing key for access tokens
JWT_REFRESH_SECRET	string ≥ 32 chars	(Reserved; refresh tokens are opaque + hashed.)
CSRF_SECRET	string ≥ 32 chars	Seeds <code>csrf-csrf</code> HMAC
ACCESS_TOKEN_TTL	duration string (default 15m)	Access JWT lifetime
REFRESH_TOKEN_TTL_DAYS	int (default 30)	Refresh token lifetime
RATE_LIMIT_WINDOW_MS	int (default 900000)	API rate-limit window
RATE_LIMIT_MAX	int (default 300)	API rate-limit max per window
AUTH_RATE_LIMIT_MAX	int (default 10)	Login/refresh max failed attempts per window
LOG_LEVEL	trace=fatal (default info)	Pino log level
SMTP_HOST / SMTP_PORT / SMTP_USER / SMTP_PASS	optional	If all four are set, used as the email transport. Otherwise the server falls back to <code>Ethereal</code> test inbox (preview URL logged per email).
EMAIL_FROM	string (default <code>EduConnect <noreply@educonnect.com.au></code>)	"From" address on enquiry emails.

5. Database schema (Prisma)

Source of truth: `server/prisma/schema.prisma` .

Enums

Role: ADMIN, UNIVERSITY, AGENT, CONSULTANT, STUDENT
AccountStatus: PENDING, ACTIVE, SUSPENDED
Tier: FREE, PREMIUM, ENTERPRISE
PostStatus: PUBLISHED, HIDDEN, REMOVED
PostCategory: SCHOLARSHIPS, VISA_TIPS, COURSES, CAMPUS_LIFE, CAREER, STUDENT_LIFE, EVENTS
MediaType: NONE, IMAGE, VIDEO
LeadStatus: NEW, CONTACTED, CONVERTED, CLOSED
CampaignStatus: DRAFT, ACTIVE, PAUSED, ENDED

Core identity

- **User** — single users table with `role` discriminator. Holds `email`, `passwordHash` (argon2id), `name`, `avatarUrl`, `status` (defaults PENDING for non-students), `lastLoginAt` . 1:1 to one of `UniversityProfile` / `AgentProfile` / `ConsultantProfile` / `StudentProfile` .
- **UniversityProfile** — `userId @id`, `shortName`, `location`, `type`, `description`, `website`, `phone`, `logoUrl`, `coverImageUrl`, `foundedYear`, `studentCount`, `internationalPct`, `ranking`, `rating`, `reviewCount`, `views`, `inquiries`, `tier`, `verified`, `tuitionMin / Max / Currency`, plus string arrays `courses`, `scholarships`, `intakes`, `facilities`, `accreditations` .
- **AgentProfile** — `contactPerson`, `phone`, `location`, `description`, `website`, `logoUrl`, `yearsExperience`, `studentsPlaced`, `partnerInstitutions`, `successRate`, `rating`, `reviewCount`, `verified`, `tier`, arrays `certifications` / `services` / `languages` / `specialisations`, `maraNumber` .
- **ConsultantProfile** — `phone`, `location`, `description`, `website`, `yearsExperience`, `studentsAssisted`, `successRate`, `rating`, `reviewCount`, `verified`, `tier`, `qualifications` / `services` / `languages` / `specialisations`, `hourlyRate` .
- **StudentProfile** — `phone`, `nationality`, `currentEducation`, `interestedIn`, `preferredLocations`, `budgetMin / Max`, `bio`, `intakeTarget` .

Auth

- **RefreshToken** — `tokenHash` (sha256 of opaque token), `userId`, `userAgent`, `ip`, `expiresAt`, `revokedAt`, `replacedBy` (chain for rotation).

Posts / social

- **Post** — `authorId`, `title`, `content`, `category`, `mediaType`, `mediaUrl`, `tags[]`, `status` (default PUBLISHED), `isPinned`, `upvoteCount`, `commentCount`, `timestamps`.
- **Comment** — `postId`, `authorId`, `text`, `upvoteCount`, `timestamps`.
- **PostVote** — composite key (`userId`, `postId`) .
- **Bookmark** — composite key (`userId`, `postId`) .

Reviews

- **Review** — `reviewerId`, `targetId`, `targetRole` (UNIVERSITY/AGENT/CONSULTANT), `rating` (1-5), `title?`, `body`, `verified` . Unique on (`reviewerId`, `targetId`) .

Enquiries / Leads

- **Lead** — `studentId`, `targetId`, `targetRole` (UNIVERSITY/AGENT/CONSULTANT), `programme?`, `message`, `status` (default NEW), `timestamps`. Indexed on (`targetId`, `status`) .

Campaigns

- **Campaign** — `universityId`, `name`, `audience`, `startDate`, `endDate`, `status` (default DRAFT), `impressions`, `clicks` .

Audit log

- **AuditLog** — `actorId?`, `action`, `entityType`, `entityId?`, `meta: Json?`, `ip`, `userAgent`, `timestamps`. Used for moderation.
-

6. Authentication & security

Token model

- **Access token:** JWT signed HS256 with `JWT_ACCESS_SECRET`. TTL = `ACCESS_TOKEN_TTL` (default 15m). Payload: `{ sub: userId, role }`.
- **Refresh token:** **opaque** 64-byte random hex string (NOT a JWT). The DB stores only the SHA-256 hash. Rotated on every refresh; reuse of a revoked token revokes the entire family for that user (theft heuristic).

Where tokens travel

- **Web SPA:** httpOnly **signed** cookies — `em_access` (path `/`) and `em_refresh` (path `/api/auth`). SameSite `strict` in prod, `lax` in dev. Cookie auth requires CSRF.
- **Mobile / API clients:** Authorization: `Bearer <accessToken>` header. Bearer-authenticated requests are immune to CSRF (browsers can't forge headers via a CSRF attack), so the server skips CSRF for them.

`attachUser` middleware (`shared/middleware/auth.js`)

Bearer header takes precedence over the cookie. If `Authorization: Bearer ...` is present, the request is marked `isBearerAuth = true` and CSRF is skipped. Otherwise the signed cookie is consulted.

CSRF (`shared/middleware/csrf.js`)

- Library: `csrf-csrf` (double-submit cookie pattern).
- Cookie name: `__Host-em.csrf` in prod, `em.csrf` in dev.
- Token header on mutating requests: `X-CSRF-Token`.
- Issued via `GET /api/auth/csrf` (idempotent; force-overwrites stale cookies).
- Skipped automatically for Bearer-authenticated requests.

Helmet / extra headers

Helmet configures CSP (`default-src 'none'`, `connect-src 'self'`, `frame-ancestors 'none'`), HSTS (1y, includeSubDomains, preload), `X-Frame-Options DENY`, `X-Content-Type-Options nosniff`, `Cross-Origin-Resource-Policy same-site`, `Referrer-Policy no-referrer`. Plus a manual `Permissions-Policy` and `Cache-Control: no-store`.

Rate limits (`shared/middleware/rateLimits.js`)

- **API limiter:** mounted on `/api`. Window `RATE_LIMIT_WINDOW_MS`, max `RATE_LIMIT_MAX`. Key by user id (falls back to IP).
- **Auth limiter:** login + refresh. `skipSuccessfulRequests: true` so only failed attempts count.
- **Signup limiter:** 5 / hour, IP-based.

Other guards

- `hpp` on every request (HTTP parameter pollution).
 - `compression`, `cookie-parser` with `COOKIE_SECRET`, body limit `100kb`.
 - Pino redacts `Authorization`, `Cookie`, `Set-Cookie`, `*.password`, `*.passwordHash`, `*.token`, `*.accessToken`, `*.refreshToken`, `*.SMTP_PASS`.
-

7. REST API reference

All paths are prefixed with `/api`. JSON in / JSON out.

Conventions

- **Auth (cookie):** the signed `em_access` cookie is sent automatically by the browser. The web client uses `credentials: 'include'` .
- **Auth (Bearer):** include header `Authorization: Bearer <accessToken>` .
- **CSRF:** for cookie-authed mutating calls (POST/PUT/PATCH/DELETE) include header `X-CSRF-Token: <token>` . Get the token from `GET /api/auth/csrf` . Bearer-authed calls skip CSRF.
- **Pagination:** `?page=1&limit=20` (max 50–100 depending on the route).
- **Errors:** `{ error: { code, message, details? } }` . `details.issues[]` is populated for zod validation failures.

7.1 Health

- `GET /api/health` — unrated. Returns `{ server, uptime, timestamp, memory, database }` . Returns 503 when Postgres is unreachable.

7.2 Auth (/api/auth)

Method & path	Auth	CSRF	Body / query	Description
<code>GET /csrf</code>	Public	—	—	Issue a CSRF token (also sets the cookie). Force-overwrites.
<code>POST /signup</code>	Public	No	<code>signupSchema</code> (discriminated by role)	Creates a user. STUDENT auto-active; UNI/AGENT/CONSULTANT start PENDING. Sets cookies + returns <code>{ user, tokens }</code> .
<code>POST /login</code>	Public	No	<code>{ email, password }</code>	Verifies argon2id hash. Constant-time on missing user. Returns <code>{ user, tokens }</code> and sets cookies. Updates <code>lastLoginAt</code> .
<code>POST /refresh</code>	Public	No	<code>{ refreshToken? }</code> (web reads cookie)	Rotates the refresh token; detects reuse and revokes the family.
<code>POST /logout</code>	Required	Yes	<code>{ refreshToken? }</code>	Revokes the presented refresh token, clears cookies.
<code>GET /me</code>	Optional	—	—	Returns <code>{ user }</code> or <code>{ user: null }</code> .

7.3 Directory (/api/{universities,agents,consultants,students})

Method & path	Auth	Role	Body	Description
<code>GET /universities</code>	Public	—	<code>listQuery</code>	Paginated list. Filters: <code>q</code> , <code>verified</code> , <code>location</code> . Sort: <code>rating</code> (default), <code>name</code> , <code>newest</code> .
<code>GET /universities/compare?ids=a,b,c</code>	Public	—	—	Batch lookup, returns 2–4 IDs preserving caller order.
<code>GET /universities/:id</code>	Public	—	—	Detail (also bumps <code>views</code> <code>async</code>).
<code>PATCH /universities/me</code>	Required	UNIVERSITY	<code>updateUniversitySchema</code>	Update own University profile.

GET /agents, GET /agents/:id, PATCH /agents/me	...	AGENT	updateAgentSchema	Same shape as universities.
GET /consultants, GET /consultants/:id, PATCH /consultants/me	...	CONSULTANT	updateConsultantSchema	Same shape.
PATCH /students/me	Required	STUDENT	updateStudentSchema	Self-update only — no public student listing.

7.4 Posts (/api/posts)

Method & path	Auth	Body / query	Description
GET /	Optional	postListQuery	Sorted by hot (default — pinned, upvotes, recent), new, or top. Includes per-viewer hasUpvoted / hasBookmarked.
GET /me	Required	—	Posts authored by the caller.
GET /bookmarks	Required	—	Posts the caller has bookmarked.
GET /:id	Optional	—	Post + comments + viewer flags.
POST /	Required	createPostSchema	Create. Author = caller.
PATCH /:id	Required	updatePostSchema	Update. Author or ADMIN only.
DELETE /:id	Required	—	Author or ADMIN only.
POST /:id/upvote	Required	—	Toggle upvote (transactional with upvoteCount).
POST /:id/bookmark	Required	—	Toggle bookmark.
GET /:id/comments	Public	—	List comments.
POST /:id/comments	Required	createCommentSchema	Add comment + bump commentCount.
DELETE /comments/:id	Required	—	Author or ADMIN only. Decrements commentCount.

createPostSchema : title (3-200), content (10-10000), category (enum), mediaType (default NONE), mediaUrl? (URL), tags[] (max 5).

7.5 Business (/api)

These all live in the business module router and are mounted at the root of /api (no extra prefix).

Method & path	Auth	Role	Body	Description
GET /campaigns	Required	UNIVERSITY	—	Caller's campaigns.
POST /campaigns	Required	UNIVERSITY	createCampaignSchema	Create campaign (refine endDate ≥ startDate).
PATCH /campaigns/:id	Required	UNIVERSITY	updateCampaignSchema	Owner only.

DELETE /campaigns/:id	Required	UNIVERSITY	—	Owner only.
GET /leads	Required	UNI/AGENT/CONSULTANT	—	Enquiries received by the caller.
GET /leads/mine	Required	STUDENT	—	Enquiries the student has submitted.
POST /leads	Required	STUDENT	createLeadSchema	Persists a Lead and emails the recipient . Bumps inquiries on UniversityProfile for university targets.
PATCH /leads/:id/status	Required	UNI/AGENT/CONSULTANT	updateLeadStatusSchema	Recipient (target) only.
GET /reviews/target/:id	Public	—	—	Reviews about a target user.
POST /reviews	Required	STUDENT	createReviewBodySchema	Target must be active UNI/AGENT/CONSULTANT (can't self-review). One review per (reviewer, target). Recomputes rating + reviewCount on the profile.

createLeadSchema : targetId (cuid string), programme? (≤ 200), message (10–2000).

7.6 Admin (/api/admin)

All endpoints require AUTH + role ADMIN.

Method & path	Body / query	Description
GET /overview	—	{ counts: { universities, agents, consultants, students, posts, pendingApprovals, leads } }
GET /users	?role=&status=&q=&page=&limit=	Paginated users with profiles.
POST /users/:id/approve	—	Sets status ACTIVE and verified=true on the profile. UNI/AGENT/CONSULTANT only.
POST /users/:id/suspend	—	Sets status SUSPENDED and revokes all refresh tokens for the user.
POST /users/:id/reactivate	—	Sets status ACTIVE.
POST /posts/:id/pin	—	Toggle isPinned.
PATCH /posts/:id/status	{ status: PUBLISHED HIDDEN REMOVED }	Set post status.

8. Email service (enquiries)

Implemented in `server/src/shared/services/email.js`. One transporter is built lazily on the first call:

- If `SMTP_HOST` + `SMTP_PORT` + `SMTP_USER` + `SMTP_PASS` are all set → real SMTP.
- Otherwise → `nodemailer.createTestAccount()` spins up an Ethereal inbox at runtime and every send is logged with a preview URL.

`sendEnquiryEmail({ to, targetName, studentName, studentEmail, programme, message })` composes a styled HTML + plain-text email. The recipient's inbox sees the student's name, programme of interest (if provided), and the message body. `Reply-To` is the student's email so a direct reply lands back to them.

The `createLead` controller awaits the DB write, then calls `sendEnquiryEmail` in a fire-and-forget `Promise.catch` that logs failures — slow SMTP never blocks the HTTP response.

9. Web client architecture

Routing (`App.jsx`)

Layered:

- `RedirectIfAuthenticated` for `/login`, `/signup` — bounces to `/` if already signed in.
- `RequireAuth` for marketplace, feed, post detail, static pages.
- `ProtectedRoute` `allowedRoles=[...]` for the role dashboards (`/admin`, `/university`, `/agent`, `/consultant`, `/student`) — wraps `<DashboardLayout />`.

`<ScrollToTop />` resets scroll on every navigation.

Auth + state (`store/useStore.js`)

Zustand store, **auth only**. Domain data is fetched per-page. Shape:

```
{
  currentUser, isAuthenticated, authReady,
  hydrate(), login(email, password), signup(payload), logout(), setCurrentUser(user)
}
```

`hydrate()` is invoked once on App mount (`/auth/me`). Roles are normalised lowercase for the existing UI (`admin` / `university` / `agent` / `consultant` / `student`).

API client (`api/client.js`)

A single fetch wrapper with:

- `credentials: 'include'` always (cookie auth).
- `X-CSRF-Token` header injected on mutating calls; token cached in module scope and refetched on demand.
- Silent refresh on 401 — calls `/auth/refresh` once and retries the original request.
- CSRF rotation on 403 + `code: CSRF_INVALID` — drops the cached token and retries once.
- Concurrent refresh / CSRF deduplication via promise singletons.
- Throws a typed `ApiError(message, { status, code, details })`.

`api.resetCsrf()` is called by login/signup/logout because the session identifier changes and the old CSRF token is no longer valid.

Endpoints (`api/endpoints.js`)

One object per server module: `authApi`, `directoryApi`, `postsApi`, `reviewsApi`, `campaignsApi`, `leadsApi`, `adminApi`. Mirrors the REST surface 1:1.

Mappers (`api/mappers.js`)

Bridges the **server** vocabulary (UPPER_SNAKE enums, nested `user.university / user.agent /etc.`) to the **UI** vocabulary (lower-kebab, flat shape):

- `ROLE_FROM_API / ROLE_TO_API`
- `CATEGORY_FROM_API / CATEGORY_TO_API`
- `MEDIA_FROM_API / MEDIA_TO_API`
- `LEAD_STATUS_FROM_API , CAMPAIGN_STATUS_FROM_API`
- `normalisePost(p) , normaliseDirectoryItem(u) , normaliseLead(l) , normaliseCampaign(c)` — pages call these after `useApiResource(...)` .

Data fetching (`hooks/useApiResource.js`)

Generic hook designed for React 19 / Strict Mode:

- `fetcher` stored in a ref → no closure-thrash on re-renders.
- `deps` hashed to a stable `depsKey` .
- Spinner only on the **initial** load — refetches keep stale data on screen so actions don't flash.
- Returns `{ data, loading, error, refetch, setData }` . `setData` is used for optimistic updates.

Notable UI

- **Marketplace detail pages** (`AgentDetail.jsx` , `UniversityDetail.jsx`) have a single **Send enquiry** CTA. Submitting writes a Lead and triggers the server-side email.
- **Student dashboard** shows a list of submitted enquiries with status (NEW/CONTACTED/CONVERTED/CLOSED).
- **University / Agent / Consultant dashboards** show received enquiries; status can be updated via `PATCH /api/leads/:id/status` .

10. Background jobs (`jobs/cron.js`)

Two jobs. Timezone Australia/Melbourne.

Job	Schedule	Action
<code>cleanupRefreshTokens</code>	<code>10 2 * * *</code>	Deletes refresh tokens that are expired or revoked > 30 days.
<code>endExpiredCampaigns</code>	<code>0 * * * *</code>	<code>Campaign.status: ACTIVE → ENDED</code> where <code>endDate < now</code> .

Started via `startCronJobs()` after `app.listen` . Stopped via `stopCronJobs()` during graceful shutdown (SIGINT / SIGTERM). The HTTP server then drains, Prisma disconnects, with a 10s force-exit fallback.

11. Validation, error handling, conventions

Zod schemas

Every mutating endpoint gets a `validate({ body | query | params })` middleware. Schemas live next to their module (`*.schema.js`). Errors are converted to a typed `BadRequestError` with `details.issues = [{ path, message }]` so the client can show field-level feedback.

Typed application errors (`shared/utls/errors.js`)

- `AppError(message, { status, code, details? })` — base class.
- `BadRequestError (400, BAD_REQUEST)`
- `UnauthorizedError (401, UNAUTHORIZED)`
- `ForbiddenError (403, FORBIDDEN)`
- `NotFoundError (404, NOT_FOUND)`
- `ConflictError (409, CONFLICT)`

- `TooManyRequestsError` (429, `RATE_LIMITED`)

Error middleware (`shared/middleware/errorHandler.js`)

Maps:

- CSRF rejection → 403 `CSRF_INVALID`
- `AppError` instance → its declared status + code
- Prisma `P2002` → 409 `CONFLICT`
- Prisma `P2025` → 404 `NOT_FOUND`
- CORS rejection (`Origin ... not allowed by CORS`) → 403 `CORS_BLOCKED`
- Anything else → 500 `INTERNAL` with stack only in non-prod

`notFoundHandler` returns 404 `NOT_FOUND` for unknown routes.

`asyncHandler`

Tiny wrapper used on every async route handler so thrown errors propagate to the error middleware:

```
asyncHandler(fn) = (req, res, next) => Promise.resolve(fn(...)).catch(next) .
```

Logging

- Pino + `pino-http`; pretty in dev.
- Auto-skip `/api/health` to avoid noise.
- Per-request: log level upgraded to `error` on 5xx, `warn` on 4xx.
- Redacted: `Authorization` , `Cookie` , `Set-Cookie` , password fields, JWTs / opaque tokens, `SMTP_PASS`.

12. Operational notes

Database migrations

```
npm run migrate          # prisma migrate dev (creates a new migration)
npm run prisma:deploy    # in CI / prod (applies pending migrations)
npm run seed             # repopulates demo data
```

Graceful shutdown

`SIGINT` / `SIGTERM` triggers cron stop, HTTP drain, and `prisma.$disconnect()` . Force-exit timer at 10s.
 Unhandled rejections / uncaught exceptions log fatal and `process.exit(1)` .

Prisma client

Singleton in `db/prisma.js` . Hot-reload-safe via `globalThis.__prisma__` .

13. Directory of all files (quick index)

Server

```
src/
  app.js, server.js
  config/env.js, config/logger.js
  db/prisma.js
  jobs/cron.js
  modules/
    index.js
    auth/{auth.controller, auth.routes, auth.schema, auth.service}.js
```

```
posts/{posts.controller, posts.routes, post.schema}.js
directory/{directory.controller, directory.routes, profile.schema}.js
business/{campaigns,leads,reviews}.controller.js
    business.{routes,schema}.js
admin/{admin.controller, admin.routes}.js
shared/
  middleware/{auth, csrf, cors, errorHandler, securityHeaders,
    rateLimits, validate}.js
  utils/{tokens, cookies, asyncHandler, password, errors}.js
  services/email.js
  validators/common.schema.js
prisma/
  schema.prisma, seed.js, migrations/*
```

Client

```
src/
  App.jsx, main.jsx
  api/{client, endpoints, mappers}.js
  store/useStore.js
  hooks/useApiResource.js
  components/
    layout/{DashboardLayout, Navbar, Sidebar, Footer}.jsx
  pages/
    auth/{Login, Signup}.jsx
    feed/(via Feed.jsx, PostDetail.jsx)
    marketplace/{Universities, Agents, Consultants,
      UniversityDetail, AgentDetail}.jsx
    admin/{AdminDashboard, ManageUniversities, ManageAgents,
      ManageConsultants, ManagePosts, AdminSettings}.jsx
    university/{UniDashboard, UniProfile, UniAnalytics, UniCampaigns}.jsx
    agent/{AgentDashboard, AgentProfile, AgentAnalytics}.jsx
    consultant/{ConsultantDashboard, ConsultantProfile,
      ConsultantAnalytics}.jsx
    student/{StudentDashboard, StudentProfile}.jsx
    dashboard/MyPosts.jsx
    static/* (about, contact, faq, blog, privacy, terms, ...)
```
